LedgerFlow on AWS

A live deployment of the LedgerFlow payments backend in AWS Mumbai (ap-south-1) on 27 October 2026: what was deployed, how it was exercised, and an annotated screenshot of each service.

What happened

- **Infrastructure:** `terraform apply` created 73 resources from `infra/terraform`: VPC, load balancer, ECS Fargate cluster with two services, RDS PostgreSQL, SNS, five SQS queues, S3, EventBridge Scheduler, Secrets Manager, CloudWatch logs, alarms and dashboard, ECR, IAM roles and the GitHub OIDC deploy role.
- **Images:** both services (`ledgerflow-api`, `mock-bank`) were built from the repo's multi-stage Dockerfile and pushed to ECR.
- **Traffic:** a k6 load test sent create + capture payments at 20 payments/second for 5 minutes through the public load balancer, against the deliberately unreliable mock bank.
- **Reconciliation:** a message on the `settlement-trigger` queue (exactly what the nightly 00:30 IST schedule sends) made mock-bank write the day's settlement file to S3, which LedgerFlow reconciled automatically.
- **Teardown:** everything was removed with `terraform destroy` afterwards. The always-on parts cost about USD 70-80/month, so the deployment is not kept running; the whole run cost well under one dollar.

Measured results on AWS

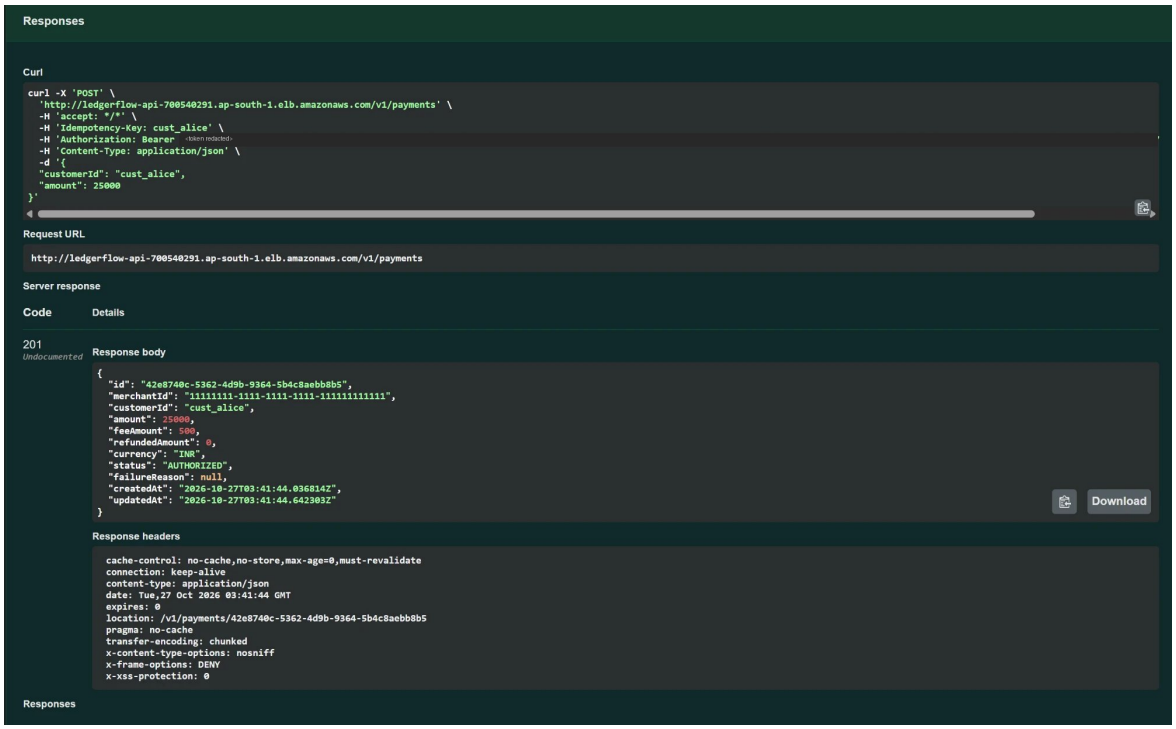
Measure	Result	Source
Requests through the load balancer (20-minute window)	9,629 total: 9,624 2xx, 3 4xx, **0 5xx**	CloudWatch ALB metrics
Server-side latency	p50 148 ms; p99 about 9 s (deliberate bank hangs + retries)	ALB TargetResponseTime
Settlement file	5,257 rows written to S3 by mock-bank	S3 settlements/2026-10-27/
Reconciliation	**5,248 matched**; 0 duplicates, 0 amount mismatches, 0 missing in bank; 9 missing in ledger (still PENDING_CONFIRMATION when the file was cut)	GET /v1/reconciliation/2026-10-27
Queues after the run	All five queues at 0 messages, both DLQs empty	SQS console
Alarms	6 alarms, all OK	CloudWatch

The 9 'missing in ledger' rows are not errors: the bank had approved those collects, but LedgerFlow was still confirming them through its status poller when the file was generated. That is exactly the kind of timing difference reconciliation exists to surface.

Request path and where each screenshot fits

Step	AWS service	Section
Merchant calls the API over HTTP	Application Load Balancer -> ECS Fargate task (ledgerflow-api)	1, 2
Container images	ECR	3
Payments, ledger, outbox, idempotency keys	RDS PostgreSQL (private subnets)	4
JWT key, webhook key, DB password	Secrets Manager	5
Payment events fan out; alarm emails	SNS (payment-events, ledgerflow-alerts)	6
Webhook delivery and settlement work	SQS with dead-letter queues	7
Settlement files in, reports out	S3	8
Nightly settlement trigger	EventBridge Scheduler	9
Metrics, dashboard, logs, alarms	CloudWatch	10, 11

1. Live API call through the load balancer (Swagger UI)



Swagger UI served by the deployed service, calling POST /v1/payments on the public ALB address. The bearer token has been redacted from the screenshot.

What the screenshot shows

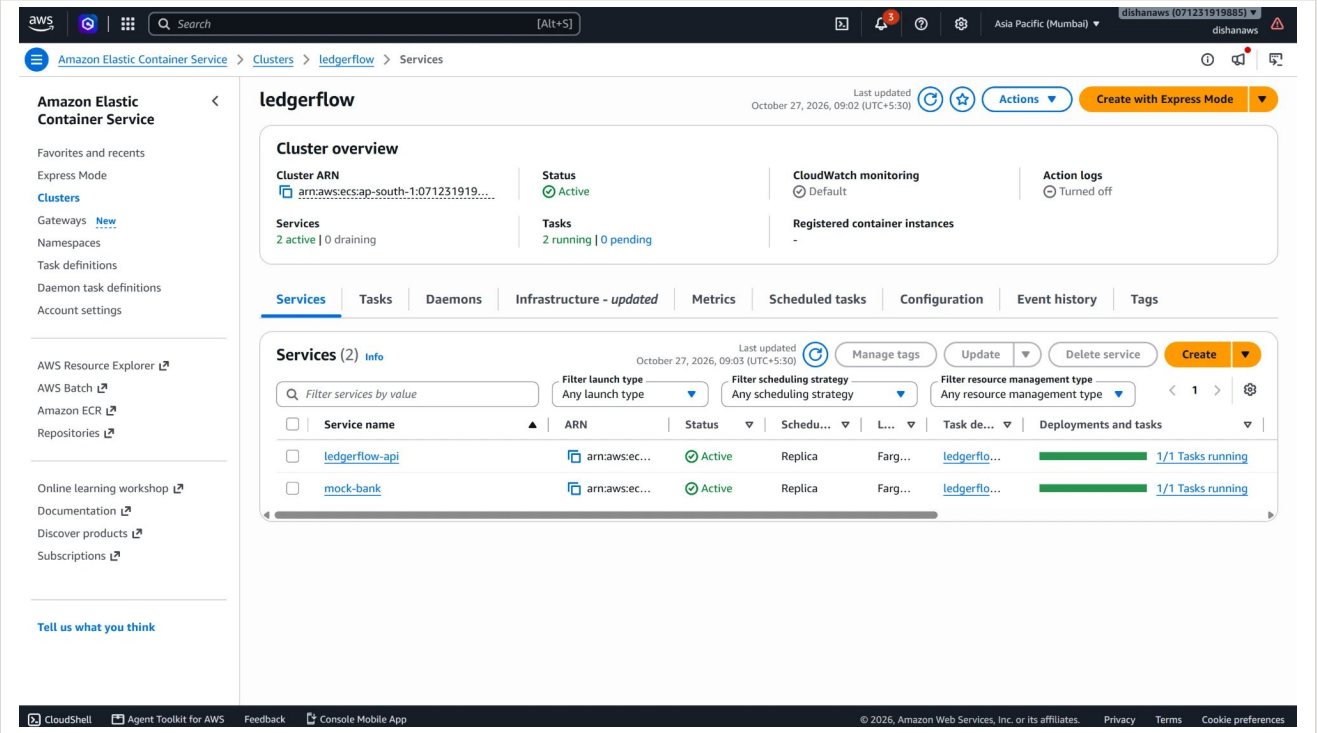
- A real request to `http://ledgerflow-api-700540291.ap-south-1.elb.amazonaws.com/v1/payments` with an `Idempotency-Key` header and a merchant JWT.
- Response **201** with a payment of 25,000 paise (Rs 250), fee 500 paise (2%), status **AUTHORIZED**: the API called mock-bank inside the VPC and recorded its approval.
- A `Location` header pointing to the new payment, and security headers (`X-Frame-Options: DENY`, `nosniff`, no-cache) added by Spring Security.

Why it is built this way

- The ALB is the only public entry point. ECS tasks accept traffic only from the ALB's security group, and the database has no internet route at all.
- There is no /dev/token endpoint in AWS (it only exists in the local profile). The token was minted with `tools/mint_token.py` from the HS256 key stored in Secrets Manager.
- Amounts are integer paise end to end, so there is no floating point anywhere in the money path.

How to say it in an interview: "This is the deployed service answering through the AWS load balancer: the payment was authorized by the bank service running next to it in ECS, and the response shows the fee calculated in integer paise."

2. ECS cluster: both services running on Fargate



Amazon ECS > Clusters > ledgerflow > Services.

What the screenshot shows

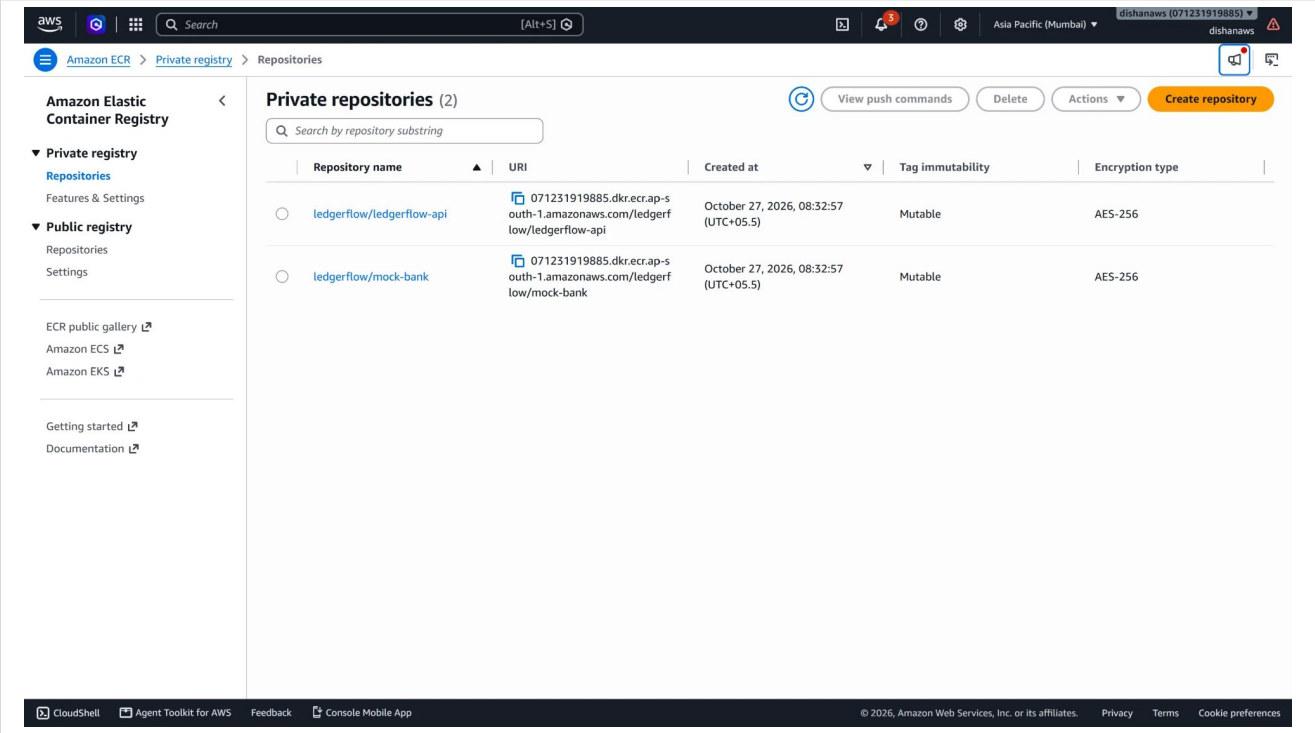
- Cluster **ledgerflow** is Active with **2 services** and **2 running tasks**, 0 pending.
- `ledgerflow-api` and `mock-bank` are both Replica services on Fargate with **1/1 tasks running**; the green bars mean the latest deployment completed.
- Each service runs a task definition created by Terraform (api: 0.5 vCPU / 1 GB; mock-bank: 0.25 vCPU / 512 MB).

Why it is built this way

- Fargate runs containers without managing EC2 instances; the same Docker images run locally in docker compose.
- Rolling deployments with the **deployment circuit breaker** enabled: if new tasks fail health checks, ECS rolls back to the previous version automatically.
- mock-bank has no load balancer. The API finds it at `mock-bank.ledgerflow.local` through Cloud Map private DNS, and its security group only allows the API.
- The API is stateless, so scaling out means raising the desired count. Background jobs (outbox relay, status poller) already share work safely across tasks with `FOR UPDATE SKIP LOCKED`.

How to say it in an interview: "Both services run as Fargate tasks behind a circuit-breaker deployment. The bank simulator is only reachable inside the VPC through Cloud Map, so it is never exposed publicly."

3. ECR: private image repositories



Amazon ECR > Private registry > Repositories.

What the screenshot shows

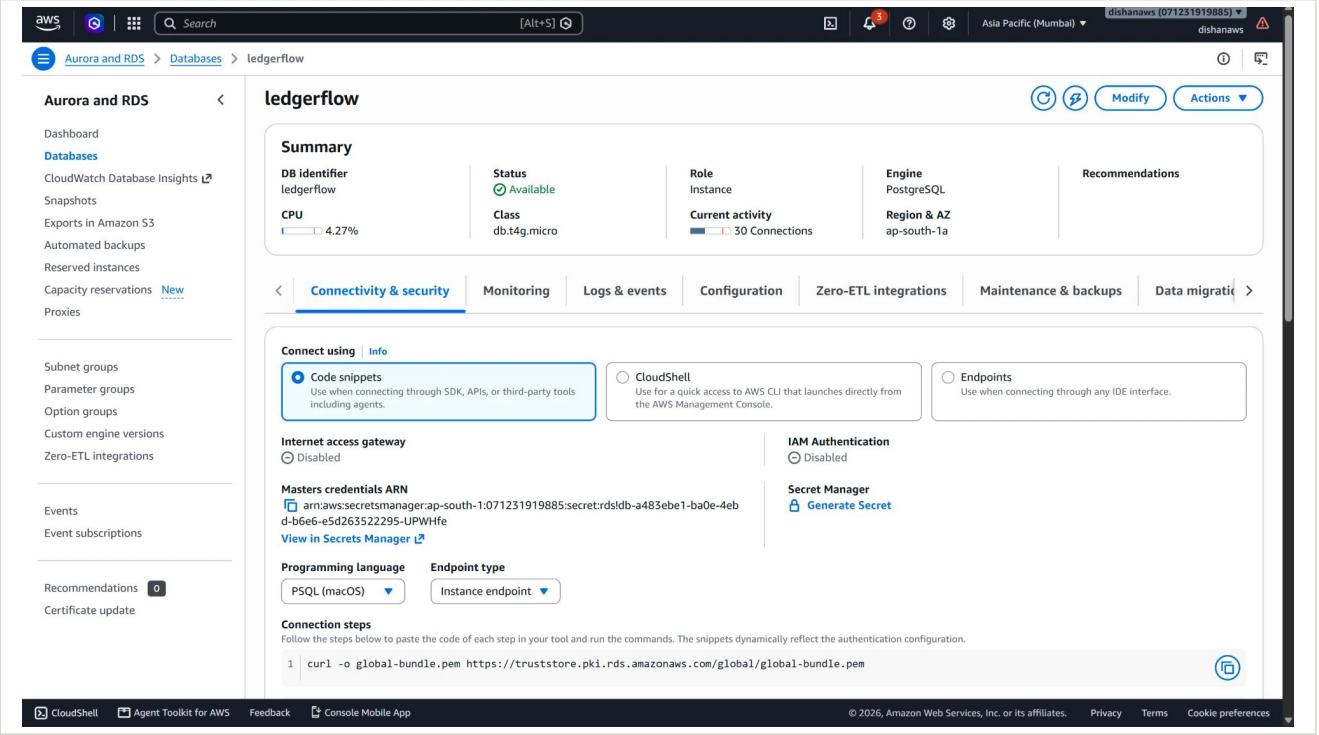
- Two private repositories, `ledgerflow/ledgerflow-api` and `ledgerflow/mock-bank`, created by Terraform.
- Images are encrypted at rest (AES-256). Tags are mutable so `latest` can move; CI also pushes an immutable tag per commit SHA.

Why it is built this way

- The registry sits in the same account and region as ECS, so pulls are fast and need no external credentials.
- A lifecycle policy keeps only the most recent images, so storage cost stays flat.
- In the CI/CD pipeline, GitHub Actions assumes an AWS role through **OIDC** (no stored AWS keys), builds both images, pushes them here and deploys the new task definitions to ECS.

How to say it in an interview: "Images live in private ECR repositories. The pipeline pushes them using short-lived OIDC credentials, so there are no long-lived AWS keys stored in GitHub."

4. RDS PostgreSQL: the ledger database



Aurora and RDS > Databases > ledgerflow > Connectivity & security.

What the screenshot shows

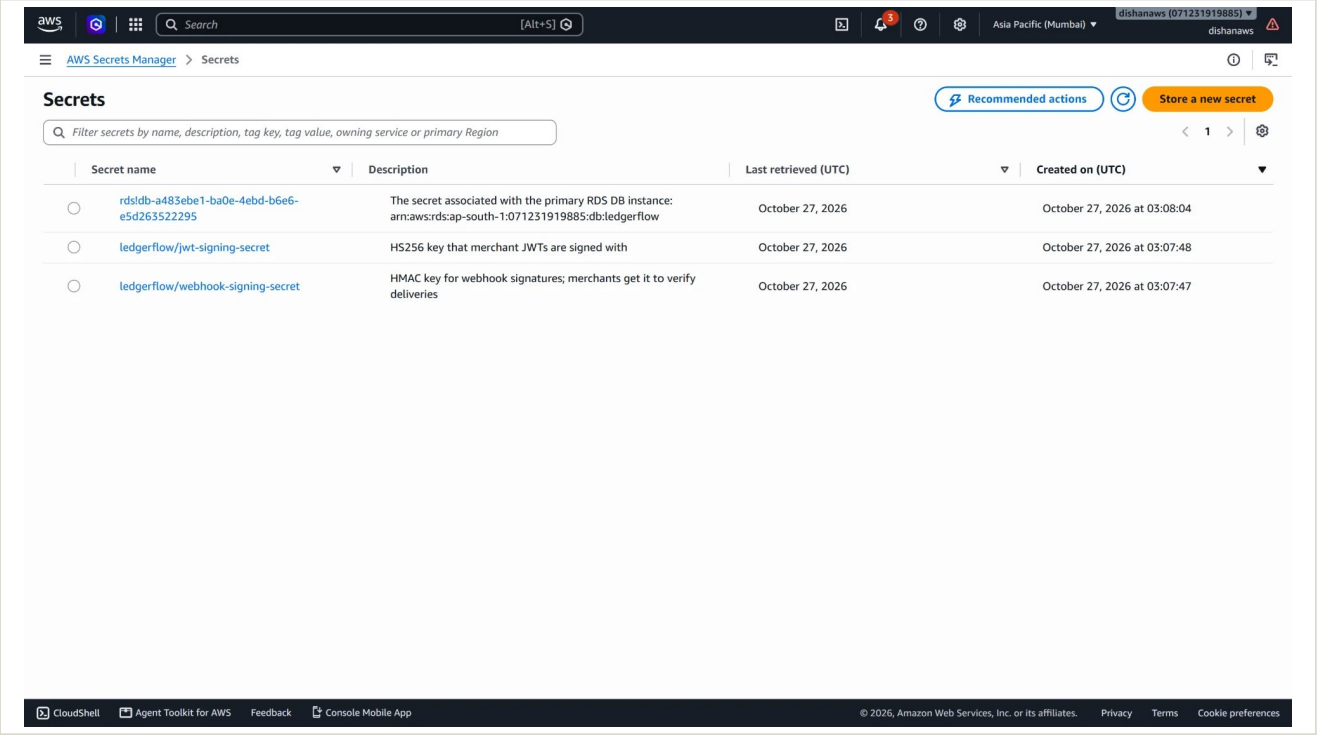
- Instance **ledgerflow**, PostgreSQL, class **db.t4g.micro** (Graviton), status **Available**, in ap-south-1a.
- **30 connections**: exactly the API's HikariCP pool size (raised from 10 to 30 after load testing showed requests queuing for connections).
- CPU 4.27% after the load test finished.
- **Internet access gateway: Disabled.** The database is reachable only from inside the VPC.
- **Master credentials ARN** points to Secrets Manager: RDS generated and manages the password itself.

Why it is built this way

- The ledger depends on PostgreSQL features: row locks (`SELECT ... FOR UPDATE`), `SKIP LOCKED` for the background workers, a deferred constraint trigger that rejects unbalanced journal entries at commit, append-only triggers, and a CHECK constraint against overdrafts.
- Flyway migrations (V1-V4) create the schema automatically when the API starts.
- Private subnets only; the security group allows port 5432 from the API tasks and nothing else.
- For this demo it is single-AZ with 1-day backups. In production: Multi-AZ, longer retention, deletion protection and RDS Proxy in front.

How to say it in an interview: "The database is private and its password was generated by RDS into Secrets Manager. The ledger rules are enforced inside Postgres too, so even a bug or a manual SQL session can't unbalance the books."

5. Secrets Manager: no secrets in code or images



AWS Secrets Manager > Secrets.

What the screenshot shows

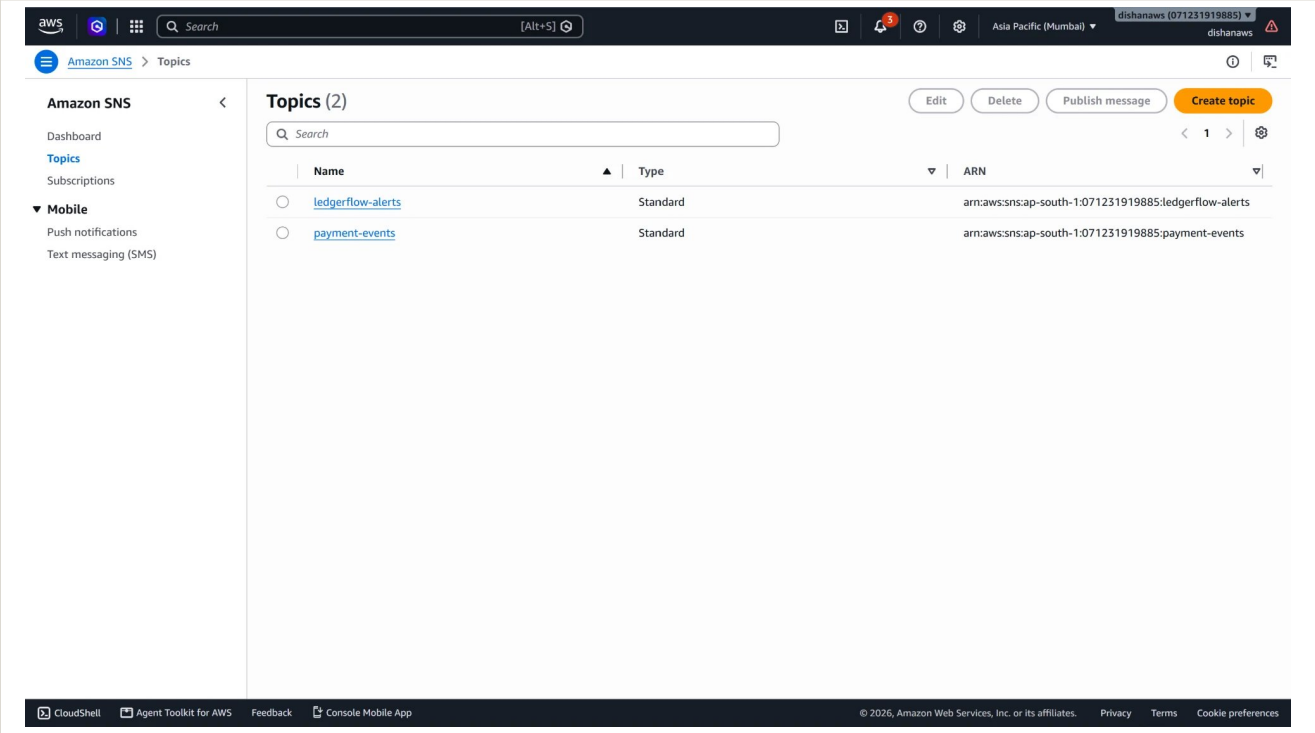
- `rds!db-...`: the database master password, created and owned by RDS.
- `ledgerflow/jwt-signing-secret`: the HS256 key merchant JWTs are signed with.
- `ledgerflow/webhook-signing-secret`: the HMAC key for webhook signatures, which merchants also get so they can verify deliveries.
- All three show 'Last retrieved' on the day, because the running services read them.

Why it is built this way

- Values are generated by Terraform (`random_password`) or by RDS, and never appear in code, Docker images or Terraform variables.
- ECS injects the DB password and JWT key into the container as environment variables at start-up, through the task execution role, which may read only these specific secrets.
- The webhook key is read by the application at runtime and cached for 5 minutes, so rotating it takes effect without a redeploy.
- Recovery window is 0 days so a teardown and re-deploy can reuse the names immediately (a demo choice).

How to say it in an interview: "Every secret lives in Secrets Manager. The database password is managed by RDS itself, and the webhook signing key can be rotated without restarting anything."

6. SNS: event fan-out and alert emails



Amazon SNS > Topics.

What the screenshot shows

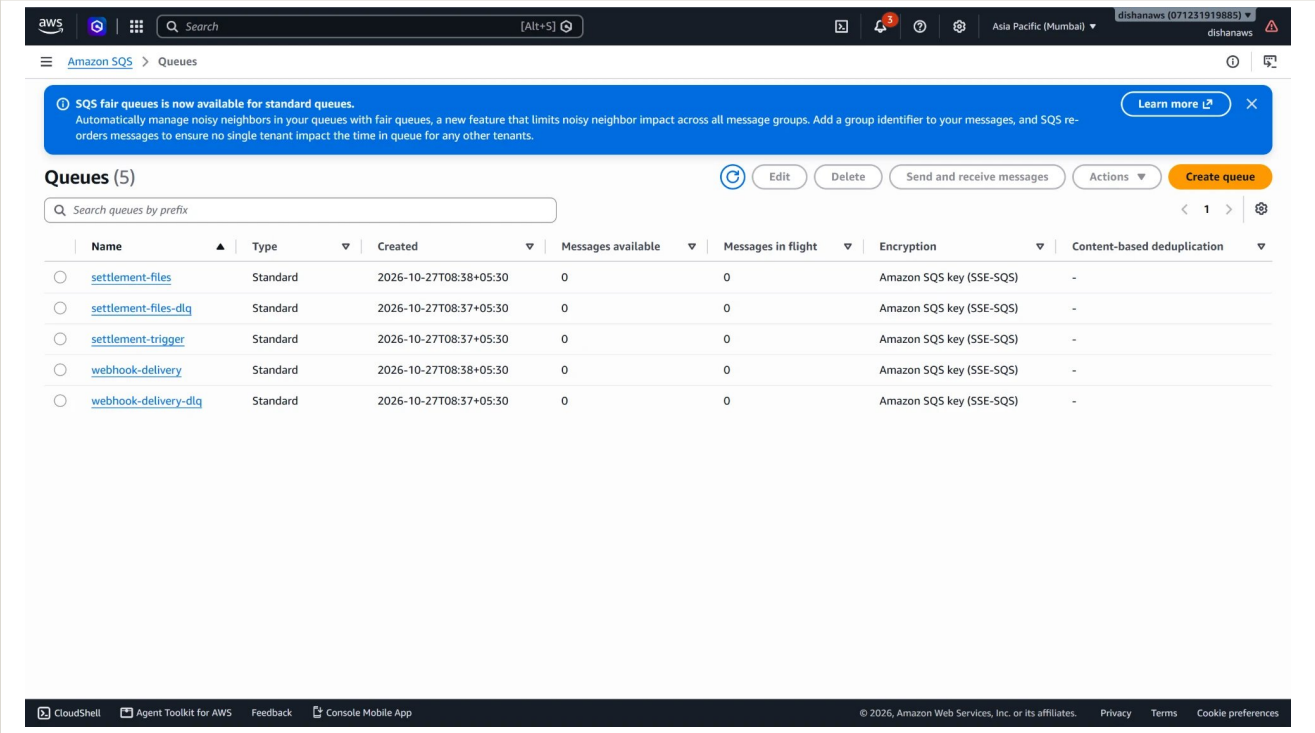
- `payment-events`: every payment state change (created, authorized, captured, refunded, settled, failed...) is published here.
- `ledgerflow-alerts`: CloudWatch alarms and notifications publish here, and it emails the address set in `terraform.tfvars`.

Why it is built this way

- **Transactional outbox:** the API writes each event to an `outbox_event` row in the same database transaction as the state change, and a relay publishes those rows to this topic in batches of 10 (`PublishBatch`). No event is lost if the app crashes, and none is announced for a change that rolled back.
- SNS gives fan-out: today the only subscriber is the webhook queue, but fraud checks or analytics could subscribe later without touching the API.
- Delivery is at least once, so consumers deduplicate by event id.

How to say it in an interview: "Events go through a transactional outbox into SNS, so the database change and the event are atomic, and new consumers can subscribe to the topic without any change to the producer."

7. SQS: work queues with dead-letter queues



Amazon SQS > Queues, after the load test had finished.

What the screenshot shows

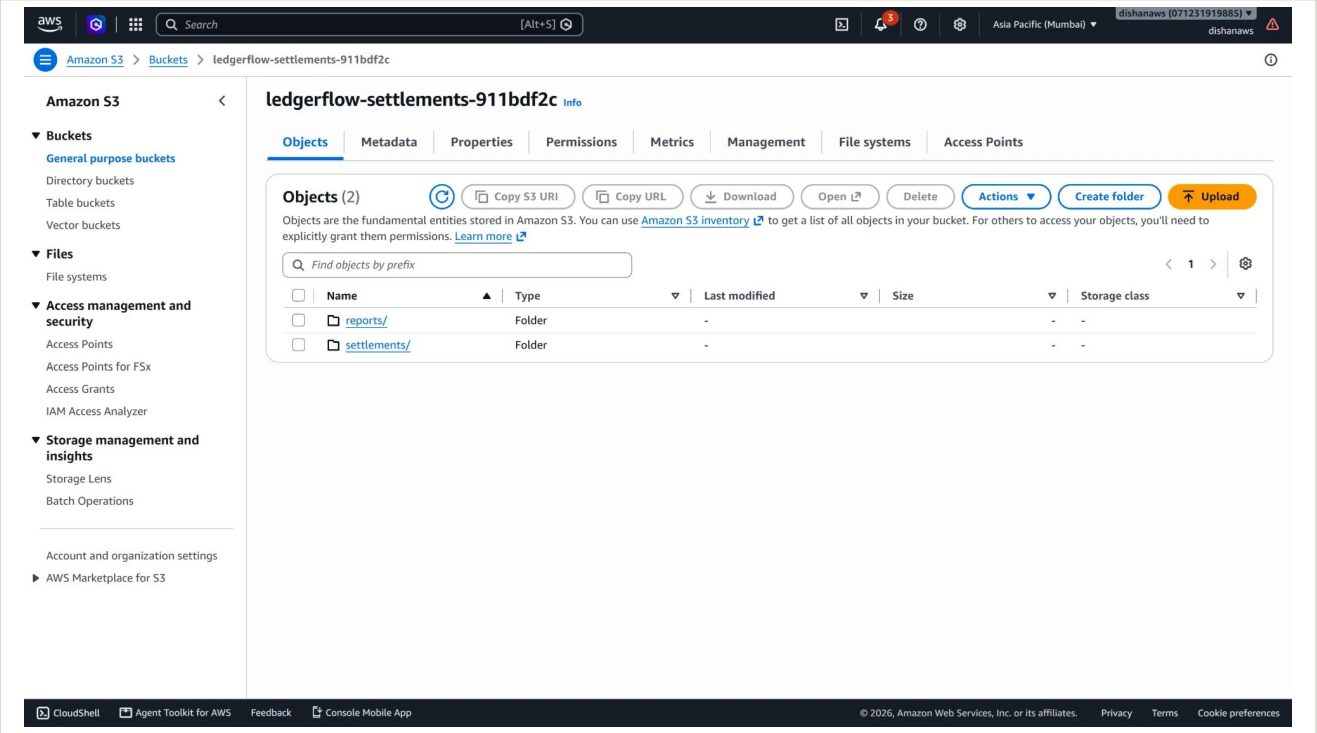
- Five standard queues, all encrypted (SSE-SQS):
- `webhook-delivery`: subscribed to the payment-events topic; the webhook dispatcher consumes it.
- `webhook-delivery-dlq`: where a webhook goes after 5 failed attempts.
- `settlement-trigger`: the nightly schedule drops a message here; mock-bank consumes it and writes the settlement file.
- `settlement-files`: receives S3 'object created' notifications; the API consumes it and runs reconciliation.
- `settlement-files-dlq`: files that could not be processed after 3 tries.
- Messages available and in flight are **0 everywhere**.

Why it is built this way

- **Retries without a scheduler:** when a webhook fails, the consumer sets the message's visibility timeout to the backoff delay (10 s, 20 s, 40 s, 80 s). SQS itself redelivers it later, so there is no retry table and no cron job.
- **Dead-lettering:** each work queue has a redrive policy (5 receives for webhooks, 3 for settlement files). A message that keeps failing moves to its DLQ, and a CloudWatch alarm fires when a DLQ is not empty.
- **Every queue at 0 messages** after 9,600+ requests means all the work was consumed and nothing ended up in a DLQ.

How to say it in an interview: "SQS is both the buffer and the retry timer: a failed webhook just becomes invisible for the backoff period, and after five tries it moves to a dead-letter queue that has an alarm on it."

8. S3: settlement files and reconciliation reports



Amazon S3 > Buckets > ledgerflow-settlements-911bdf2c.

What the screenshot shows

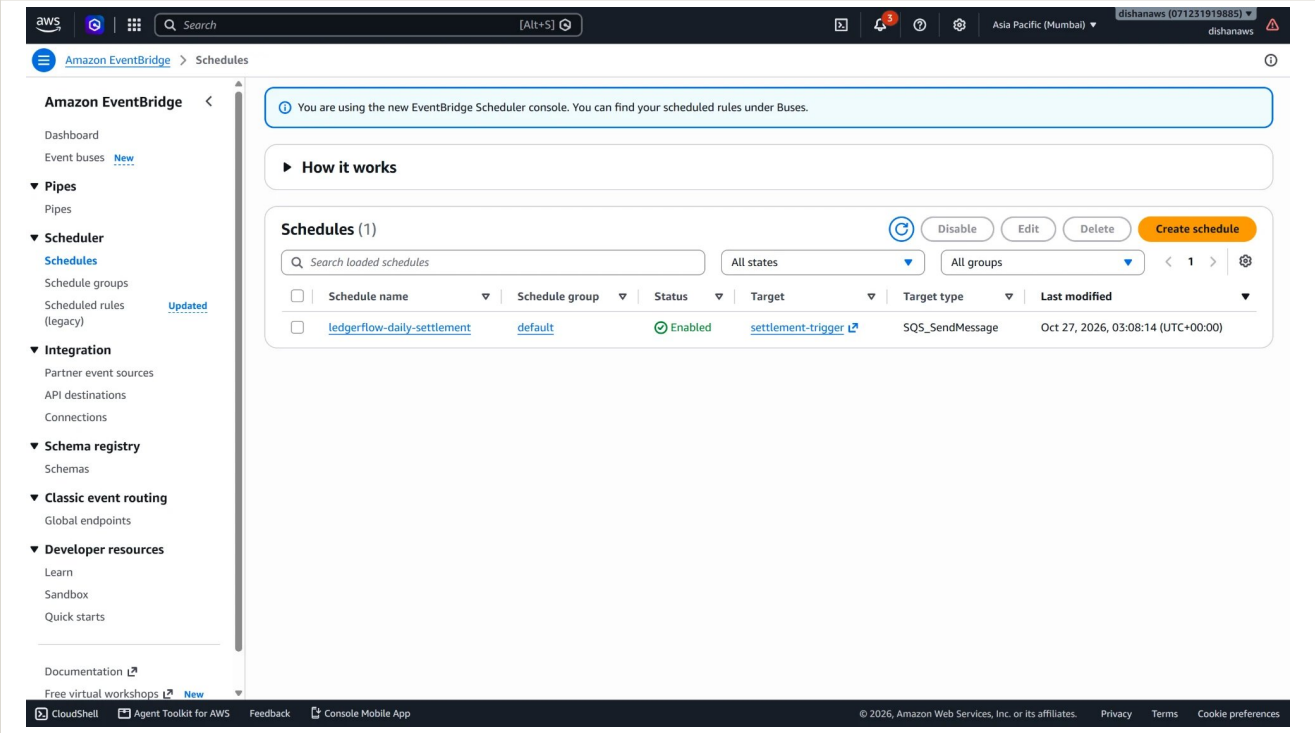
- `settlements/`: `settlements/2026-10-27/settlement.csv`, the bank's 5,257-row settlement file written by mock-bank.
- `reports/`: `reports/2026-10-27.json`, the reconciliation report LedgerFlow wrote back (5,248 matched, 9 missing in ledger).
- The bucket name has a random suffix because S3 bucket names are global.

Why it is built this way

- S3 is cheap, durable file storage and can emit an event when a file arrives. An **ObjectCreated** notification on `settlements/*.csv` goes to the `settlement-files` queue, so reconciliation starts as soon as the file lands, with no polling.
- The bucket blocks all public access, encrypts objects, and expires them after 90 days.
- IAM is per prefix: mock-bank may only write `settlements/*`; the API may only read `settlements/*` and write `reports/*`.
- The report is also stored in PostgreSQL and served to operations staff at `GET /v1/reconciliation/{date}`, which needs a token with the `ops` scope.

How to say it in an interview: "The bank drops its file into S3, S3 notifies a queue, and the reconciler runs automatically. On AWS it matched 5,248 payments and flagged 9 the bank had approved while we were still confirming them."

9. EventBridge Scheduler: the nightly trigger



Amazon EventBridge > Scheduler > Schedules.

What the screenshot shows

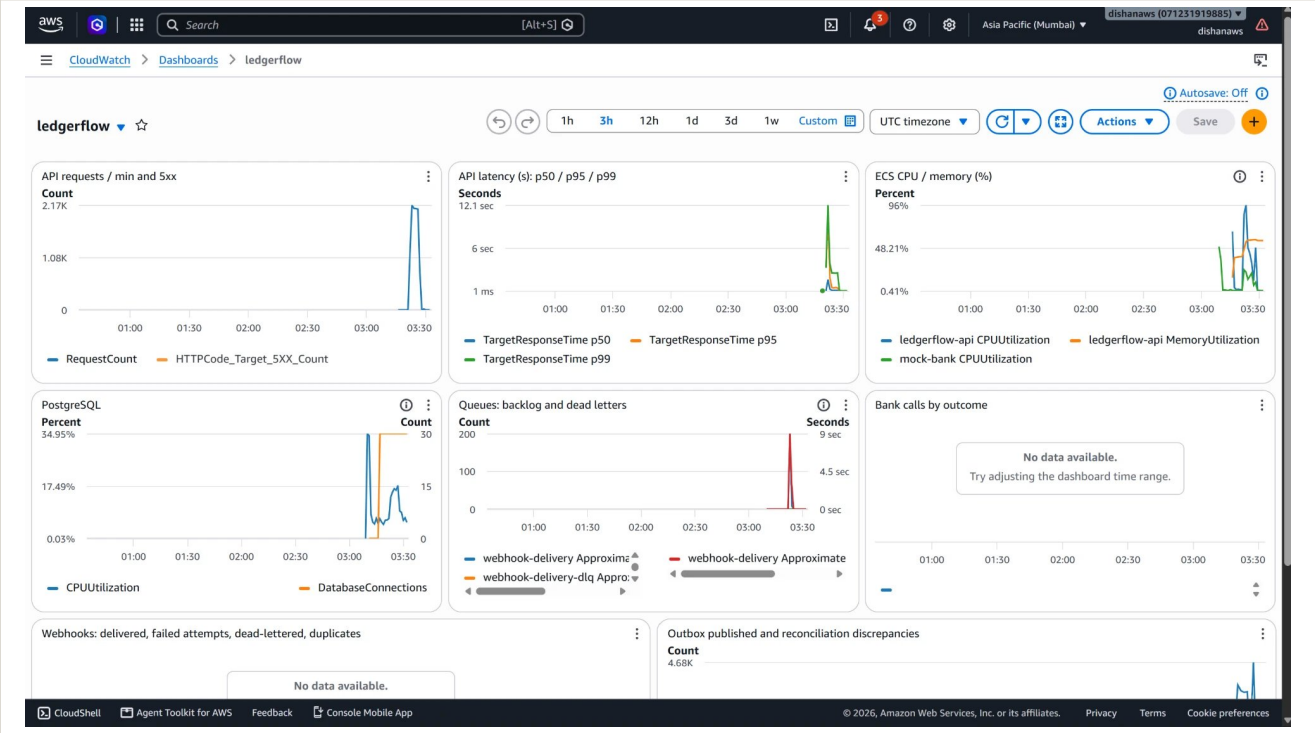
- Schedule **ledgerflow-daily-settlement**, Enabled, in the default group.
- Target: the `settlement-trigger` SQS queue (target type SQS_SendMessage).
- Cron `cron(30 0 * * ? *)` in time zone **Asia/Kolkata**: every night at 00:30 IST.

Why it is built this way

- A managed, time-zone-aware cron with nothing always-on to run it. Business dates in LedgerFlow are IST, so the schedule is too.
- It only sends a message; mock-bank decides what to do with it. That keeps the schedule decoupled from the service, and makes a manual run trivial: for this demo the same message was sent by hand to settle the current day.
- Scheduler assumes a dedicated IAM role that may only send to this one queue.

How to say it in an interview: "A managed scheduler kicks off the nightly settlement in IST by putting a message on a queue, so the schedule and the service are decoupled, and I can trigger the same flow manually."

10. CloudWatch dashboard: the load test as it happened



CloudWatch > Dashboards > ledgerflow, 3-hour view in UTC. The load test ran at about 03:22-03:27 UTC.

What the screenshot shows

- **API requests / min and 5xx:** a spike to about 2,170 requests per minute during the k6 run; the 5xx line stays at zero.
- **API latency p50 / p95 / p99:** p50 stays low. The p99 spikes (up to about 12 s at the start) are mock-bank's deliberate hangs plus the API's timeout-and-retry, which is the behaviour being tested.
- **ECS CPU / memory:** API CPU peaks during start-up and the load test; memory around 50% of 1 GB.
- **PostgreSQL:** CPU peaks around 35%; connections at 30, the size of the connection pool.
- **Queues:** the webhook queue briefly holds about 200 messages, then drains to 0; the oldest message is only seconds old; the DLQs stay empty.
- **Outbox published:** about 4,700 events per minute relayed to SNS at the peak.
- **Bank calls** and **Webhooks** show 'No data' (explained below).

Why it is built this way

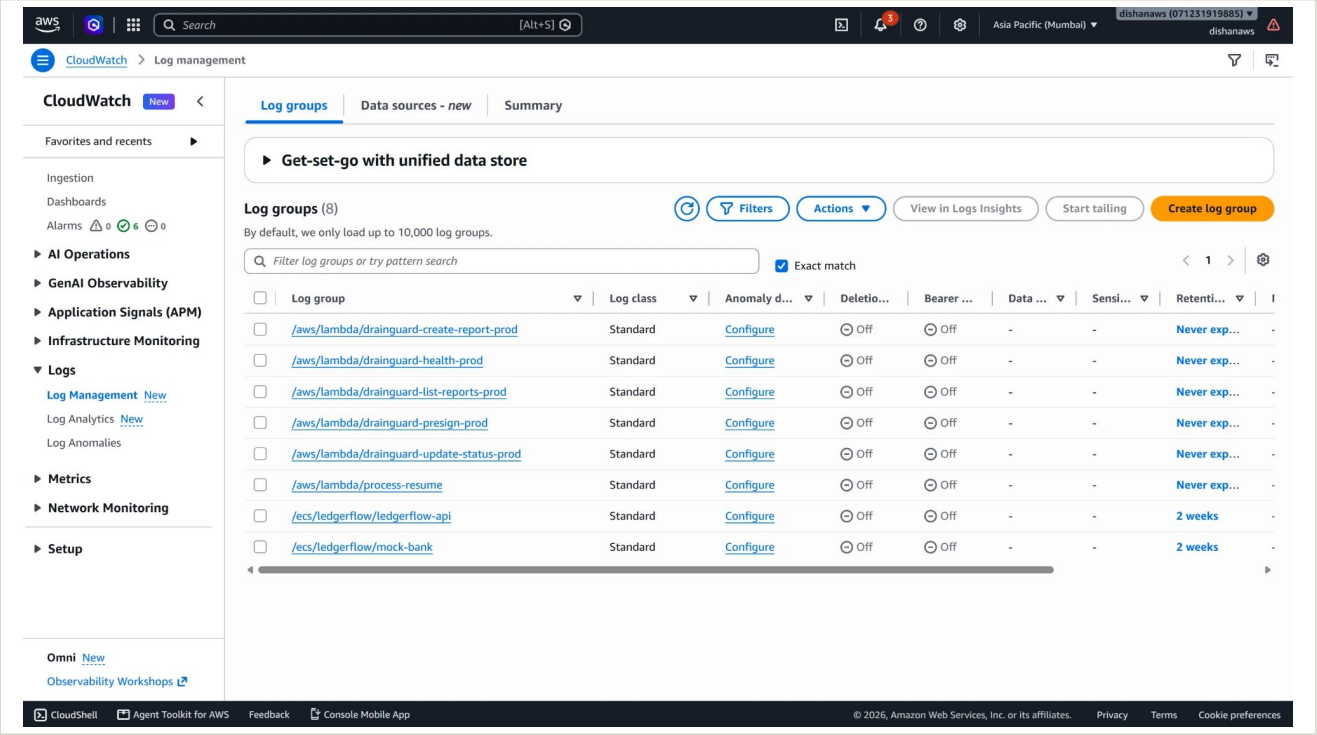
- One page mixes AWS metrics (ALB, ECS, RDS, SQS) with the application's own Micrometer metrics, published to CloudWatch under the `LedgerFlow` namespace.
- The same metrics drive six alarms (DLQ not empty, API 5xx, no healthy targets, RDS CPU, RDS storage) that email the alerts topic.

Honest notes on the two empty panels

- **Webhooks:** the demo merchant in the AWS seed data has no webhook URL, so deliveries were recorded as SKIPPED and no delivered/failed counters were emitted. Webhook retries and the DLQ are demonstrated in the local stack and the integration tests instead (98.9% delivered to a merchant failing 30% of requests).
- **Bank calls:** this panel's search expression only matched metrics without dimensions, but the bank-call counter is tagged by operation and outcome. The expression has since been changed to match on namespace and metric name; that fix has not yet been checked against a live deployment.

How to say it in an interview: "During the test the API took about 2,000 requests a minute with zero 5xx errors, the database stayed under 35% CPU, and the webhook queue spiked and drained back to zero."

11. CloudWatch Logs and alarms



CloudWatch > Log Management > Log groups. Other log groups in the list belong to unrelated projects in the same AWS account.

What the screenshot shows

- `/ecs/ledgerflow/ledgerflow-api` and `/ecs/ledgerflow/mock-bank`: one log group per service, created by Terraform with **2-week retention**.
- In the left menu: **Alarms 0 in alarm, 6 OK, 0 insufficient data**. Everything stayed healthy during the test.

Why it is built this way

- Containers log to stdout; the ECS `awslogs` driver ships it to CloudWatch, so there is no log agent to run.
- Logs are structured JSON (Elastic Common Schema format), so they can be filtered by field in Logs Insights.
- Retention is bounded so log storage cost cannot grow forever.
- The six alarms cover what an on-call engineer would care about: webhooks or settlement files stuck in a DLQ, API 5xx errors, no healthy API tasks behind the load balancer, and RDS CPU and storage.

How to say it in an interview: "Each service logs structured JSON to its own CloudWatch log group, and six alarms cover dead-letter queues, 5xx errors, unhealthy targets and the database. All six stayed green through the load test."

Cost and teardown

The deployment ran for under an hour. The always-on components would cost roughly the following per month if left running, which is why it is torn down after the demo and run locally against LocalStack the rest of the time:

Component	Approx. USD / month	Note
Application Load Balancer	18	Hourly charge plus capacity units
ECS Fargate (2 tasks)	25	0.75 vCPU and 1.5 GB in total
RDS db.t4g.micro + 20 GB	15	Single-AZ
Public IPv4 addresses	14	ALB and tasks; the trade-off for having no NAT gateway
SNS, SQS, S3, Secrets Manager, CloudWatch, Scheduler	a few	Pay per use; cents at demo volume
Total	**about 70-80**	A NAT gateway (not used) would add about 35

Reproducing this deployment

- `cd infra/terraform`, copy `terraform.tfvars.example` to `terraform.tfvars` and set the alert email.
- `terraform apply` (about 10-15 minutes, mostly RDS), then push both images to ECR, either through the GitHub Actions deploy job (set the `AWS_DEPLOY_ROLE_ARN` repository variable) or by hand with docker build/push.
- Mint a token with `tools/mint_token.py` using the JWT secret from Secrets Manager, then call the API on the `api_url` output.
- `terraform destroy` removes everything (ECR repositories, S3 bucket and secrets are configured to delete cleanly).